



Forecasting Household Energy Consumption

This project looks at time series forecasting to analyze residential energy use. Using machine learning, we develop models for short-term consumption prediction and detect usage patterns.

Problem Overview & Opportunity

Problem Statement

Smart meters/sensors produce massive datasets daily from household consumption. Most households lack visibility into consumption patterns and usage drivers. Energy dynamics can also vary due to weather, appliances, and seasonal behaviors

Solution

Data science offers predictive insights to help save costs by understanding these complex factors. We can use analytics to understand energy patterns and predict energy consumption

What Can This Solution Achieve?

Households

Save energy bills through behavior change.

Gain visibility on which appliances and conditions drive high usage.

Utilities

Better demand forecasting reduces grid strain significantly.

More effective time-of-use pricing and strategic planning capabilities.

Environmental

Reduce unnecessary electricity consumption across residential sectors.

Support sustainable, low-carbon lifestyles through data-driven insights.

Data Foundation-Sprint 1

Dataset Overview

UCI DataSet

- Measurements of electric power consumption in one household with a one-minute sampling rate over a period of almost 4 years.
- This dataset contains 2M+ measurements gathered in a house located in Sceaux (7km of Paris, France) between December 2006 and November 2010 (47 months).
- Time Series 1-minute resolution data, with 6 features
 - Voltage, Current and Global Active Power usage (Target) used for the HouseHold
 - 3 Sub-metering power features: kitchen, laundry, HVAC systems

Weather DataSet

- Historical hourly weather data from Open-Meteo API for France (2006-2010)
- 34K rows with 1 hour averaged data
- Contains 12 weather related features:
 - Temperature, Dewpoint, Relative humidity
 - surface pressure, cloudcover
 - windspeed, windgust, wind direction
 - shortwave, direct, diffuse, radiation

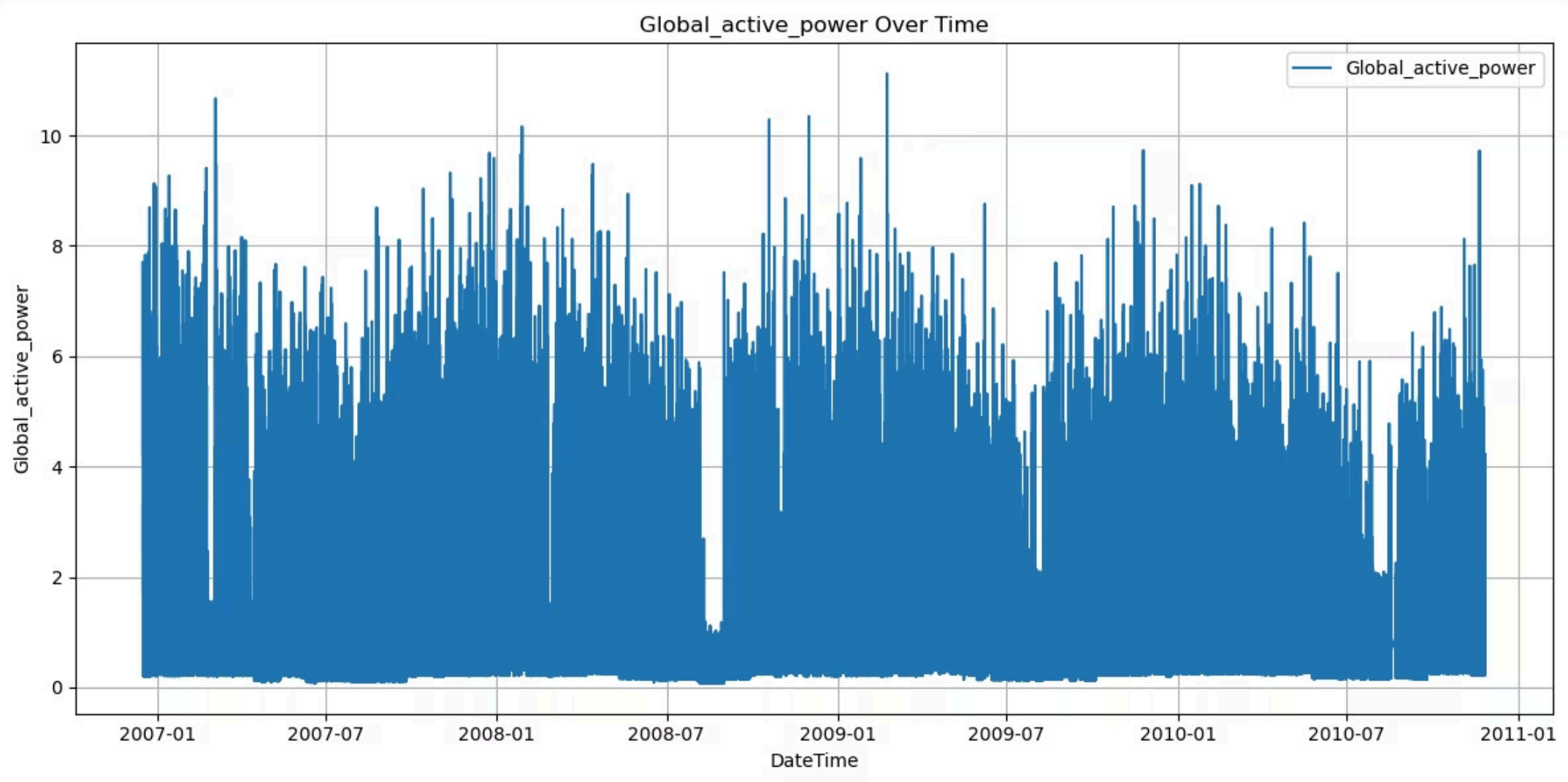
Preprocessing Procedures

Our data preparation involved several critical steps to ensure quality and usability:

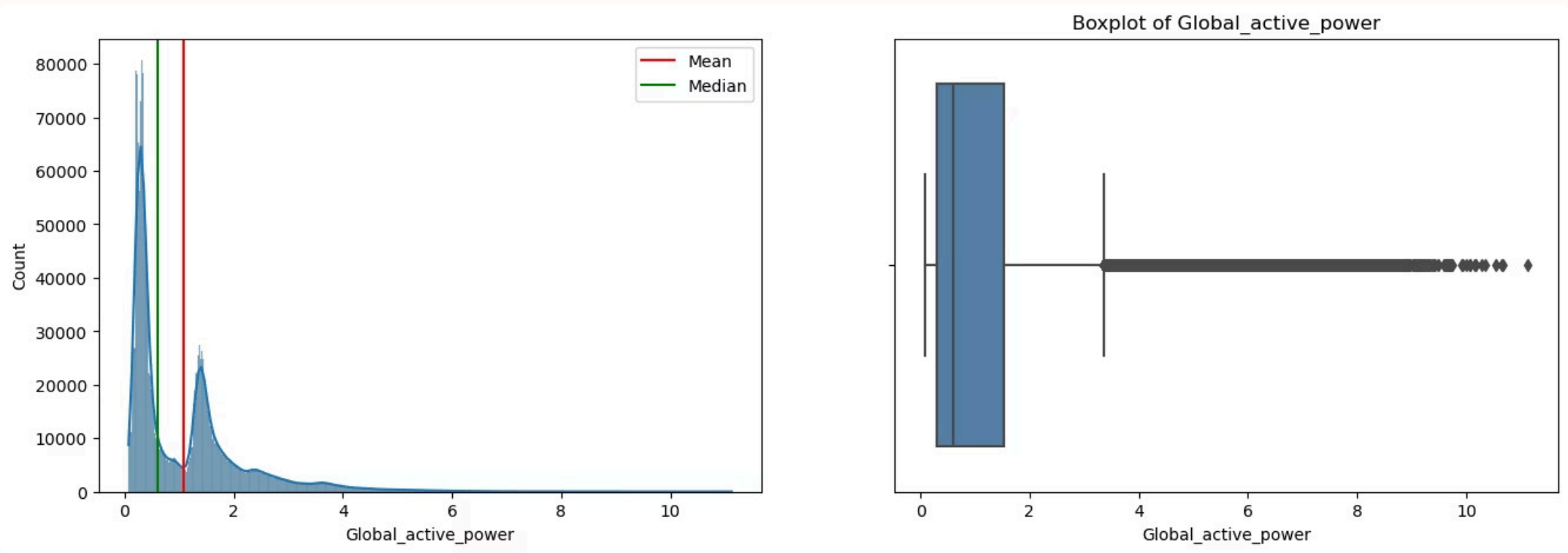
- **Merging Two Datasets:** Integrated both datasets together and resampled the weather data to minute average.
- **Missing Value Treatment:** The UCI dataset contained some missing values in the measurements **~1.25% of the rows**. All calendar timestamps are present in the dataset but for some timestamps, the measurement values were missing. These missing values were from **April 28, 2007**. They were dropped from the dataset, as were not significant compared to the number of rows.
- **Duplicated Data:** Both datasets did not contain any duplicated values.

Key Discoveries from Exploratory Data Analysis -Sprint 1/2

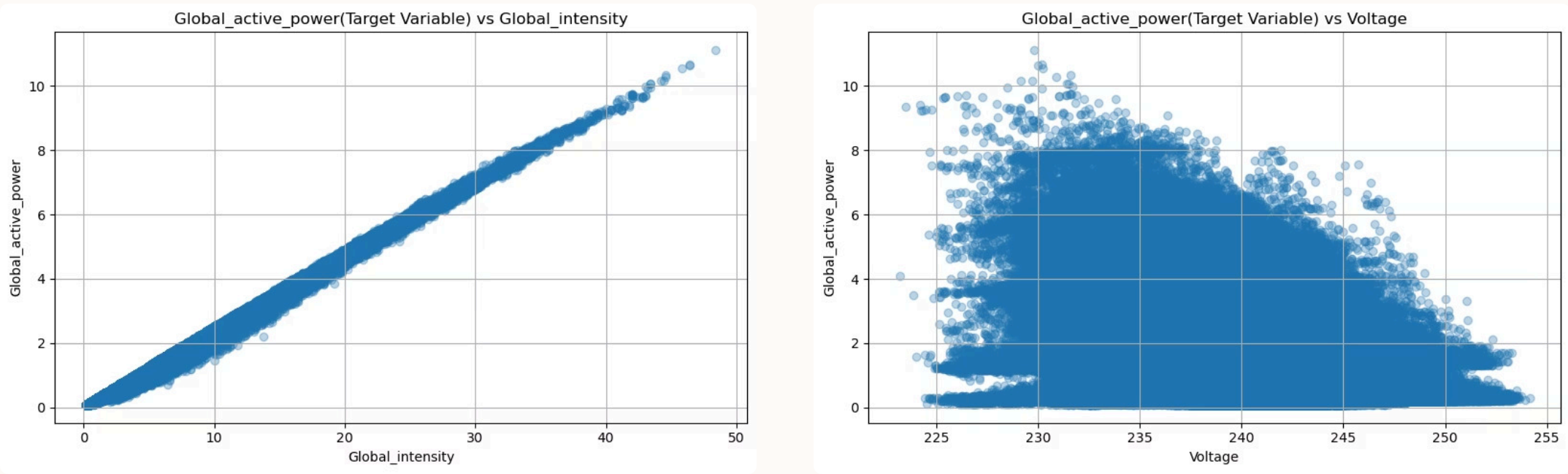
- All features are very noisy as they are sampled every minute, need to consider upsampling the data to **daily average or monthly average**



- The target variable (Power) is **not normally distributed** — it's highly skewed with heavy tails. **Outliers** are frequent; they may represent unusual events or faulty readings. The **mean > median** confirms positive skew.



- High positive correlation between **Target Variable (Power)** and **Intensity (Current)**, this makes sense as **Power= Current * Voltage**, but similar trend was not observed for Voltage



Feature Engineering -Sprint 2

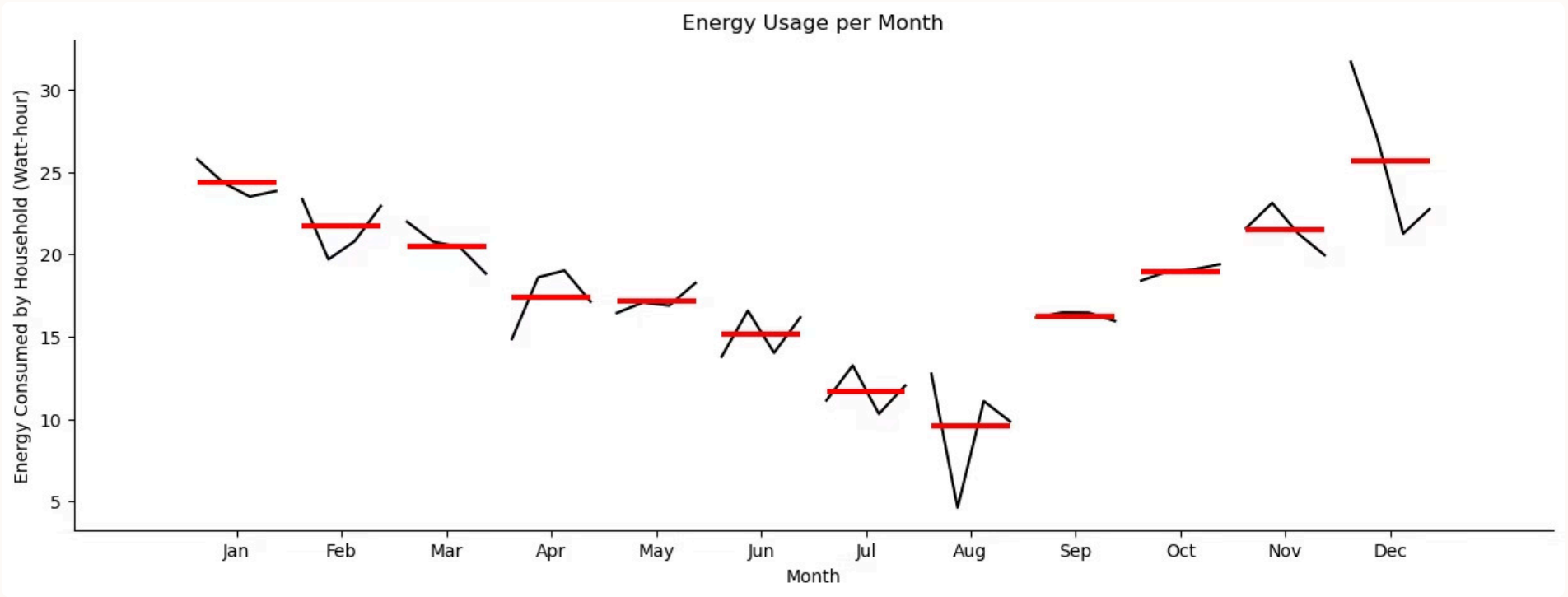
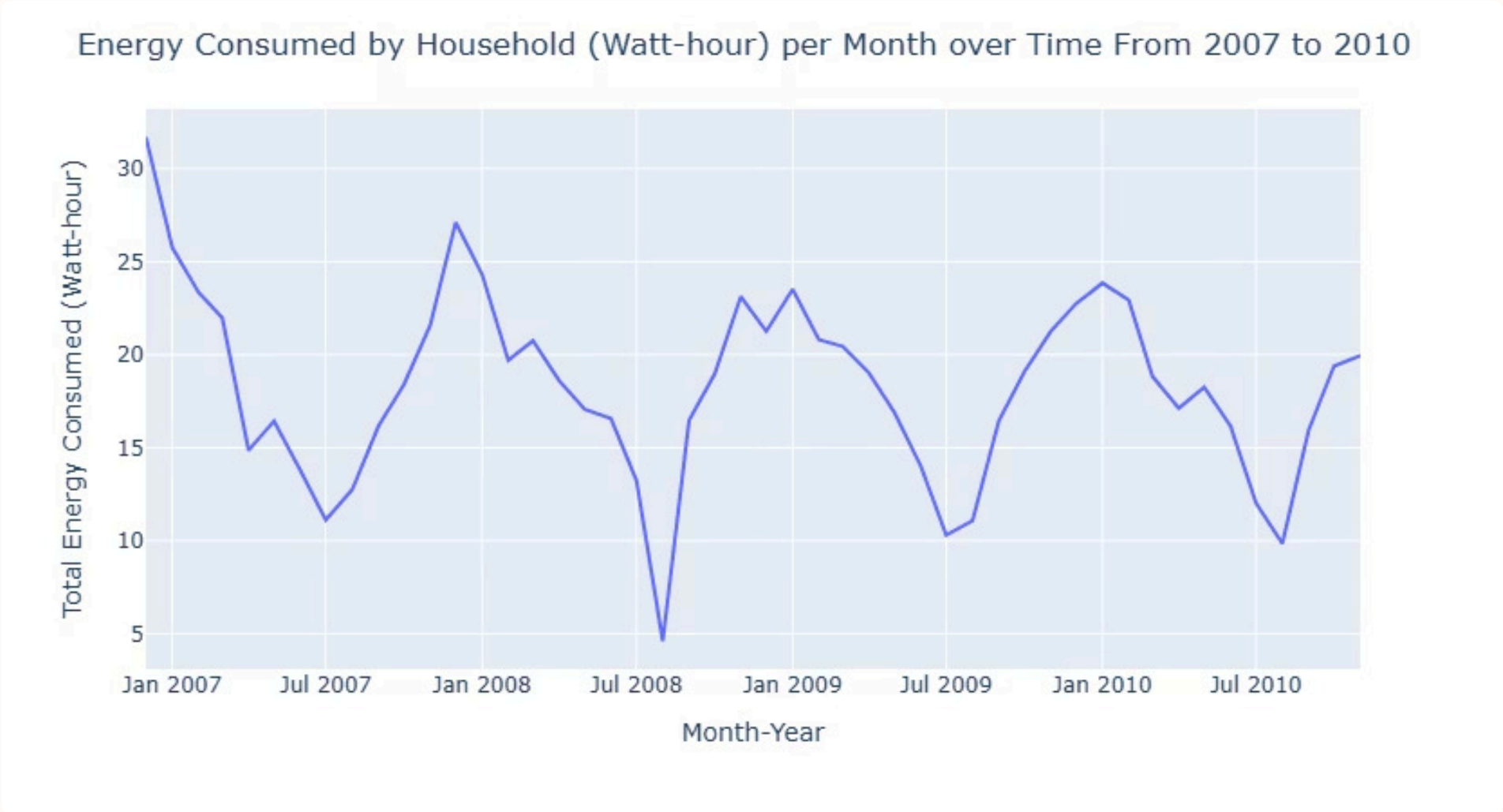
Feature Engineering:

- Created 1 derived feature capturing the **total energy consumed every minute (in watt hour) in the household by electrical equipment not measured in sub-meterings 1, 2 and 3:**
 - **Global Active power*1000/60 - sub_metering_1 - sub_metering_2 - sub_metering_3**
- Performed Unit conversions to change **Power units (kW) to Energy units (kwh)** for features to achieve consistency among submetering 1,2,3 and power variables.
- Resampled the data to **daily, week and monthly** averages to find insights about aily, weekly and sesonal patterns from dataset
- For Linear Regereession model, created a new feature **"Days from Start"**, which resembled the the total days for the 4 years spaced squentially from start month to end month

Advanced EDA Insights-Sprint 2

Seasonal and Monthly Patterns

Identified key monthly and sesonal patterns, where the energy usage is cyclic in nature decreases during summer (July, August) and increases during winter (Jan-Dec).



The red bars shows the average for the given month across all years, while each black line represent an individual month and the progression of values across the years.

Trend Sesonal Decomposition (TSD):



The trend is generally decreasing over time, indicating a long-term decrease in energy consumption by the household. The seasonal component shows a clear pattern of higher usage in winter months and lower usage in summer months, which is consistent with the expectation that households use more energy for heating in winter. The residuals show some random fluctuations around zero, indicating that there are no patterns left after accounting for trend and seasonality.

Baseline Modeling Approach- Sprint 2



Data Splitting

75% training, 25% testing with time-based separation to prevent data leakage



Model Competition

5 candidate algorithms evaluated:
Mean/Average Model, Linear Regression
Model with One Feature, Linear Regression
Model with One Hot Encoder, Linear
Regression Model with multiple dataset
features



Evaluation Metrics

Since we are predicting a continuous numeric variable, time series forecasting is fundamentally a regression problem. Therefore, we need a metric that gives us a sense of the average error of our predictions, so R^2 tends to be less useful in this case. Instead, I have employed the **Mean Absolute Error (MAE)** and the **Mean Absolute Percentage Error (MAPE)** to evaluate and compare our models.

Here is an overview of the models that I will build:

- 1. **Average Model:** $y = mean(y)$
- 2. **Linear Model with One Feature:** $y = \beta_0 + \beta_1 DaysFromStart$
- 3. **Linear Model with Dummy Variables:** $y = \beta_0 + \beta_1 DaysFromStart + \beta_2 January + \beta_3 February + + \beta_{13} December$

4. **Linear Model with Features:** $y = \beta_0 + \beta_1 X1 + \beta_2 X2 + \beta_3 X3 + + \beta_{19} X19$

where, X represents the features we have in the dataset for power df

Baseline Results

Model	Train_MAE	Test_MAE	Train_MAPE	Test_MAPE
Mean?Average	4.034347e+00	3.101882e+00	2.922166e-01	2.158319e-01
linreg	4.077902e+00	3.336767e+00	2.946289e-01	2.017694e-01
linreg_ohe	1.315639e+00	1.717609e+00	9.581909e-02	1.047659e-01
linreg_features	2.021206e-14	1.986290e-14	1.224942e-15	1.145575e-15

Linear Regression with data set features, is the current best model fitted so far

Advanced Modelling-Sprint 3

ARIMA

For our advanced modeling phase, we implemented the Autoregressive Integrated Moving Average (ARIMA) model, a widely used statistical method for time series forecasting. ARIMA models are particularly well-suited for datasets with trends and seasonality, common characteristics of energy consumption data.

Understanding ARIMA

ARIMA stands for:

- AR (Autoregressive):** This component indicates that the model uses the relationship between an observation and a number of lagged observations (i.e., past values) to predict future values. It's denoted by the parameter 'p'.
- I (Integrated):** This part signifies that the raw observations have been differenced one or more times to make the time series stationary (i.e., constant mean, variance, and autocorrelation over time). The differencing order is denoted by the parameter 'd'.
- MA (Moving Average):** This component represents the dependency between an observation and a residual error from a moving average model applied to lagged observations. It's denoted by the parameter 'q'.

Initially, an ARIMA(1,1,1) model was implemented, representing a simple autoregressive term, one differencing step, and one moving average term:



After initial evaluation, a more extensive grid search was performed to optimize the (p,d,q) parameters. The optimized ARIMA model was determined to be ARIMA(8,1,1). This suggests that the model benefits from considering more past observations (8 lags) to capture the underlying patterns in the energy consumption data, while still maintaining one differencing step and one moving average error term.



Model Performance Analysis: ARIMA(1,1,1) vs. Optimized ARIMA(8,1,1)

Comparing the two ARIMA models based on their respective plots (assuming these visuals represent forecast accuracy, e.g., predicted vs. actual values):

- The initial ARIMA(1,1,1) model likely provided a baseline forecast, capturing some of the time series patterns but potentially exhibiting larger deviations from actual values.
- The optimized ARIMA(8,1,1) model, with its increased 'p' value, is expected to show a closer fit to the actual data. This improvement indicates that considering a longer history of past values significantly enhances the model's ability to predict future energy consumption more accurately. The grid search successfully identified that more autoregressive terms were beneficial for this specific dataset.

XGBoost for Energy Consumption Forecasting

Following the ARIMA model, we implemented **XGBoost (eXtreme Gradient Boosting)**, a powerful and efficient ensemble machine learning method. XGBoost sequentially builds weak prediction models (decision trees), with each new model correcting errors from previous ones. This gradient boosting approach excels at handling complex non-linear relationships and integrating diverse features, making it highly effective for time series forecasting, especially with external factors.

Key Feature Engineering

To leverage XGBoost's capabilities, we engineered a rich set of predictive features:

- Lag Features:** Past energy consumption values (e.g., previous hour, 24 hours, 7 days) to capture temporal dependencies.
- Temporal Features:** Derived from timestamps, including hour, day, month, year, and binary indicators for holidays/weekdays.

Hyperparameter Optimization

A thorough hyperparameter optimization process was undertaken using time-series cross-validation. Key parameters tuned included `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, and regularization parameters (`reg_alpha`, `reg_lambda`) to prevent overfitting and maximize predictive power.

The initial XGBoost model served as a baseline:



Model Performance Analysis: Baseline vs. Optimized XGBoost

The baseline model showed inherent strength but exhibited deviations during volatile periods. After rigorous hyperparameter tuning, the **optimized XGBoost model** (second image) achieved significantly improved performance. This enhancement is visible as a closer alignment between predicted and actual values, with reduced error margins and better capture of peak demands and troughs.



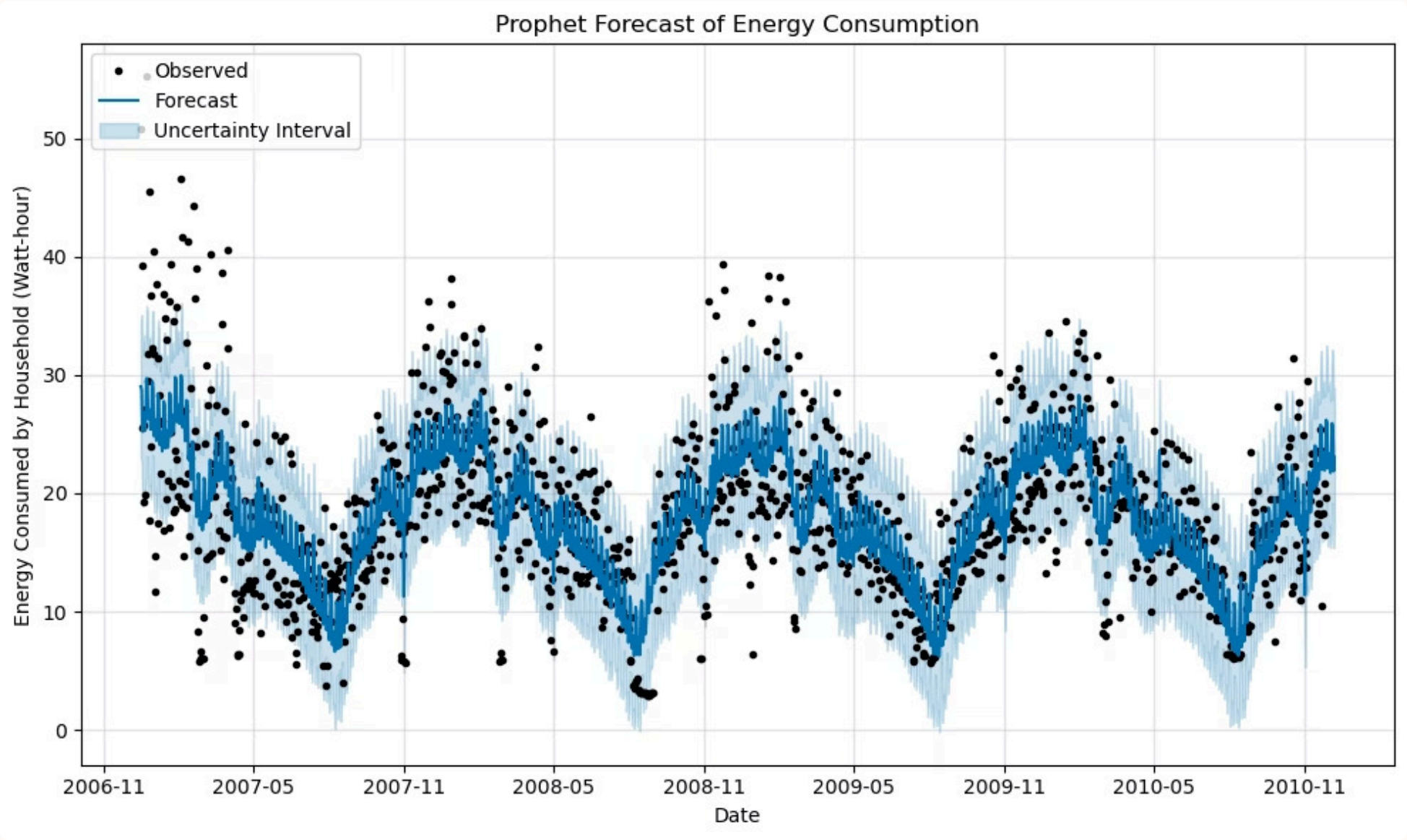
Why ARIMA Might Underperform Compared to XGBoost ?

While ARIMA is a strong contender for time series forecasting, it often performs worse than models like XGBoost, especially for complex datasets like energy consumption, due to several key factors:

- **Feature Engineering:** XGBoost (and other tree-based models) can readily incorporate a wide array of exogenous variables (e.g., temperature, humidity, time of day, day of week, holidays) as features. ARIMA's ability to handle such external predictors is limited (though ARIMAX exists, it's often less flexible). Energy consumption is heavily influenced by these external factors.
- **Non-Linear Relationships:** XGBoost, being a gradient boosting algorithm, is highly effective at capturing complex, non-linear relationships and interactions between features, which are abundant in energy consumption patterns. ARIMA, fundamentally a linear model, struggles with such complexities.
- **Robustness to Outliers:** Tree-based models can be more robust to outliers and noise in the data compared to ARIMA, which can be sensitive to unusual observations impacting its parameter estimation.
- **Handling of Data Structure:** XGBoost can work directly with a flattened dataset where time series aspects are captured through engineered lag features, while ARIMA requires strict time series assumptions like stationarity.

Facebook's Prophet for Energy Consumption Forecasting

Facebook's Prophet is a free, easy-to-use forecasting tool. It's built for business data that repeats over time (like daily or yearly patterns) and includes special events like holidays. Because of its smart design, Prophet is great for predicting things like energy use, which can be complicated. It offers a straightforward and adjustable way to forecast, unlike older, more traditional methods.

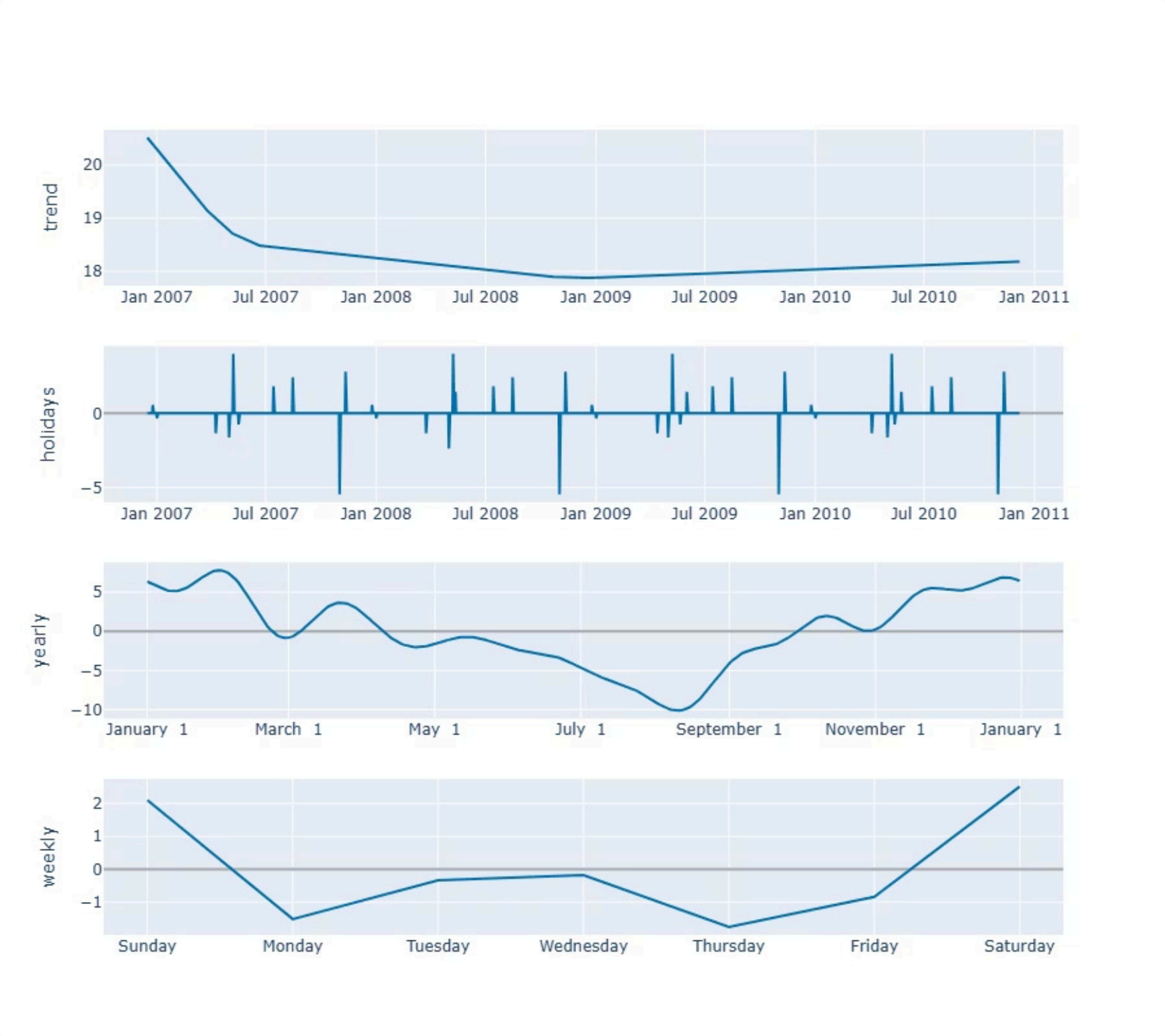


Understanding Prophet: An Additive Model

Prophet uses a simple "add-on" model with three main parts that work together:

- **Trend:** This part tracks long-term changes that don't repeat regularly. Think of it as the overall upward or downward slope. It can follow straight lines or S-shaped curves, and it automatically finds points where the trend changes.
- **Seasonality:** This captures patterns that repeat at regular times, like daily, weekly, or yearly cycles. It uses a mathematical trick (called Fourier series) to spot several repeating patterns all at once.
- **Holidays and Events:** You can tell Prophet about specific holidays or big events, and it will learn how they usually affect your data.

The model's formula is: $y(t) = g(t) + s(t) + h(t) + \epsilon(t)$.



How It Compares & What We Learned

Prophet often performs well when forecasting energy use, especially when there are clear repeating patterns and holiday effects. It tends to be better than ARIMA because it handles multiple seasonal patterns and custom holidays more easily.

Compared to XGBoost, Prophet focuses on breaking down the forecast into trend, seasonal, and holiday parts. XGBoost is great at finding hidden, complex connections and using many outside factors (like temperature). However, Prophet can do just as well, or even better, if the main things driving the forecast are strong time-based patterns and you don't need to add a lot of extra "feature engineering" (preparing outside data)

Prophet Model Insights:

- **Strong Fit to Historical Data:**

The blue forecast line closely tracks the observed black dots, indicating the model effectively captures historical energy consumption patterns.

- **Clear Seasonality:**

Regular peaks and troughs highlight pronounced seasonal cycles in household energy use.

- **Uncertainty Representation:**

The shaded region (uncertainty interval) expands in certain periods, reflecting increased model uncertainty—especially for future or volatile intervals.

- **Trend Stability:**

The overall trend remains steady over time, suggesting the underlying process is stable aside from seasonal fluctuations.

Modeling Summary

Results

	Train_MAE	Test_MAE	Train_MAPE	Test_MAPE
Mean	4.034347e+00	3.101882e+00	2.922166e-01	2.158319e-01
linreg_ohe	1.315639e+00	1.717609e+00	9.581909e-02	1.047659e-01
linreg_features	2.021206e-14	1.986290e-14	1.224942e-15	1.145575e-15
arima_111	3.826463e+00	5.577666e+00	2.330022e-01	3.979674e-01
arima_811	3.062481e+00	1.874828e+00	1.890975e-01	1.220753e-01
xgboost	4.008336e-04	1.089735e+00	2.342684e-05	5.827498e-02
xgboost_optimized	9.082587e-04	9.997100e-01	5.873678e-05	5.559898e-02
prophet	4.538755e+00	3.328523e+00	3.118428e-01	2.482262e-01

Key Insights:

- The **linreg_features** model shows near-zero error on both train and test sets, indicating probable data leakage or target-feature overlap rather than true predictive power.
- **Optimized XGBoost** achieves the second lowest Test MAE (0.0010) and a competitive Test MAPE (5.56%), marking it as the most accurate overall.
- **Linear regression with one-hot encoding (linreg_ohe)** delivers decent generalization, with a Test MAE of 1.72 and Test MAPE of 10.48%.
- The **mean model** serves as a naive baseline; any model beating its 3.10 MAE and 21.58% MAPE offers genuine predictive value.
- **Traditional ARIMA and Prophet** time-series approaches lag behind tree-based methods in this context.

Summary



Developed robust models to accurately forecast household energy consumption, aiming to optimize resource allocation, reduce operational costs, and enhance grid stability.



Data & Methodology

Leveraged historical consumption data combined with external variables (e.g., weather, holidays). Employed comprehensive EDA, feature engineering, and compared ARIMA, Prophet, and XGBoost models.



Key Findings

- Pronounced daily, weekly, and annual seasonal patterns identified.
- Strong correlation between energy use and external factors like temperature and holidays.



Model Performance

Optimized XGBoost emerged as the superior model, achieving the lowest Test MAE and MAPE (0.9997 and 5.56% respectively). Prophet performed well for seasonality, but XGBoost excelled with complex feature interactions.



Business Value

- Improved forecasting accuracy supports better energy procurement and demand-side management.
- Contributes to significant operational efficiencies and cost reductions.
- Provides critical insights for strategic infrastructure planning.



Next Steps

- Implement the optimized XGBoost model into production for real-time forecasting.
- Establish a continuous monitoring and retraining pipeline for model adaptability.
- Explore integration with real-time data sources and advanced anomaly detection techniques.

Energy Consumption DashBoard Demo

Thank You